

TP - Session 14 : Introduction a Redux

Compteur avec Redux Classique (sans Toolkit)

Module M204 - Developpement Front-End

Objectif : Créer un compteur en utilisant Redux classique pour comprendre les concepts fondamentaux (Store, Actions, Reducers).

Mise en Place du Projet

```
# Créer le projet
npm create vite@latest tp-redux-counter -- --template react
cd tp-redux-counter

# Installer Redux et React-Redux
npm install redux react-redux

# Lancer le serveur
npm run dev
```

Structure a creer :

```
src/
  redux/
    actions/
      counterActions.js
    reducers/
      counterReducer.js
      index.js
    store.js
  components/
    Counter.jsx
  App.jsx
  main.jsx
```

1 Exercice 1 : Créer les Actions (4 pts)

Fichier : `src/redux/actions/counterActions.js`

Créer les **types d'actions** (constantes) et les **action creators** pour :

- `INCREMENT` : Incrementer de 1
- `DECREMENT` : Decrementer de 1
- `INCREMENT_BY` : Incrementer d'un montant (payload)
- `RESET` : Remettre a zero

2 Exercice 2 : Créer le Reducer (5 pts)

Fichier : `src/redux/reducers/counterReducer.js`

Créer un reducer qui :

- A un **state initial** : `{ count: 0 }`

- Gere les 4 types d'actions (INCREMENT, DECREMENT, INCREMENT_BY, RESET)
- Retourne **toujours un nouvel objet** (immutabilite)
- A un case default qui retourne le state inchangé

3 Exercice 3 : Combiner et Creer le Store (3 pts)

3.1 Combiner les Reducers

Fichier : src/redux/reducers/index.js

```
import { combineReducers } from 'redux';
import counterReducer from './counterReducer';

const rootReducer = combineReducers({
  counter: counterReducer
});

export default rootReducer;
```

3.2 Creer le Store

Fichier : src/redux/store.js

Creer le store avec :

- createStore de redux
- Le rootReducer
- Support pour Redux DevTools

4 Exercice 4 : Connecter Redux a React (2 pts)

Fichier : src/main.jsx

- Importer Provider de react-redux
- Importer le store
- Wrapper <App /> avec <Provider store={store}>

Code a completer

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { Provider } from 'react-redux';
import store from './redux/store';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    { /* Ajouter le Provider ici */ }
    <App />
  </React.StrictMode>
);
```

5 Exercice 5 : Composant Counter (6 pts)

Fichier : src/components/Counter.jsx

Creer un composant qui :

- Utilise useSelector pour lire state.counter.count

- Utilisez `useDispatch` pour obtenir la fonction `dispatch`
- Affiche la valeur du compteur
- A 4 boutons : +1, -1, +5, Reset
- Chaque bouton `dispatch` l'action correspondante

Structure du composant

```
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, incrementBy, reset }
  from '../redux/actions/counterActions';

function Counter() {
  // Lire le state
  const count = useSelector((state) => state.counter.count);

  // Obtenir dispatch
  const dispatch = useDispatch();

  return (
    <div style={{ textAlign: 'center', padding: '50px' }}>
      <h1>Compteur Redux</h1>
      <h2 style={{ fontSize: '5rem', color: '#764ba2' }}>{count}</h2>

      <div style={{ display: 'flex', gap: '10px', justifyContent: 'center' }}>
        <button onClick={() => dispatch(increment())}>+1</button>
        { /* Ajouter les autres boutons */ }
      </div>
    </div>
  );
}

export default Counter;
```

Validation avec Redux DevTools

Testez votre application :

1. Installez l'extension **Redux DevTools** (Chrome/Firefox)
2. Ouvrez les DevTools (F12) et allez dans l'onglet "Redux"
3. Cliquez sur les boutons et vérifiez que :
 - Les actions apparaissent dans la liste
 - Le state change correctement
 - Le "time travel" fonctionne (cliquez sur les actions passées)

Bonus (+3 pts) :

- Ajouter une action `MULTIPLY` qui multiplie par un nombre
- Ajouter un historique des opérations (dans le state)
- Styler le composant avec CSS ou Bootstrap